

Nokoroa 技術面談 想定問答集

コードを実際に読んだ技術面接官が突いてくる箇所を、聞かれる確率 × 答えられないと痛い度で並べたもの。

ビジネス要件ではなく、コードとしての論理的整合性だけで答える。

各項目の構成:

- 該当箇所 — `file:line`
 - 回答 — トレードオフを明示した技術的な説明
 - 根拠 — 公式ドキュメントの URL、または実装・実測で確認した事実
 - 追い質問と返し — さらに掘られたときの想定
-

この資料の使い方（3つの原則）

1. 弱点は自分から先に言う

未対応の箇所を隠すと「知らなかった」に見える。先に言えば「知ったうえで優先度を判断した」に変わる。

本資料では 未対応項目を ◆ で明示しているので、該当する質問が来たら必ず自分から口に出す。

2. 「なぜそう書いたか」ではなく「何と何を天秤にかけたか」を答える

面接官が見ているのは選択の妥当性ではなく、選択肢を把握していたか。

「A にした」より「A と B を比較して、この条件では A を取った。B に倒れる条件はこれ」の方が強い。

3. 事実は実測で裏を取る

「たぶん動く」は答えにならない。本資料の技術的主張は、公式ドキュメントの引用か、実際に実行して確認した結果のいずれかに紐づけてある。

結論から言える3行

- SQL は安全。ただし書き方は指摘どおりだったので直した — `$queryRawUnsafe` でもパラメータ化はされていたが、Prisma 公式が `$queryRaw` を推奨しており *Unsafe を選ぶ理由が実際には無かった。実 DB で `::vector` キャストが動くことを確認してタグ付きテンプレートへ移行済み。

- RAG は「AI に投げた」ではなく「制約から設計した」 — pgvector の 2,000 次元上限から埋め込み次元を、`hnsw.ef_search` の既定値から検索上限を、モデルのトークン上限から入力長を決めている。
- 未対応を把握している — localStorage の JWT、トークン失効機構、CSP、Redis ストレージ。いずれも「なぜ後回しにしたか」と「正しい直し方」を言える。

第1部 — 最優先（ほぼ確実に聞かれる）

Q1. `$queryRawUnsafe` でもパラメータ化できますよね。書き換えて「安全になった」というのは嘘では？

該当箇所: `nokoroo-backend/src/embeddings/embeddings.service.ts:60, 82, 110` / コミット `9c2f78f`

回答

嘘です。正確には「元の実装も安全だった」が正しい。`$queryRawUnsafe(sql, ...values)` は第2引数以降を `$1, $2` にバインドするので、PostgreSQL に送られる Bind メッセージは `$queryRaw` と同一でした。

書き換えた理由は実行時の安全性ではなく 検証コストの局所化です。`*Unsafe` は「第1引数の文字列がどう組み立てられたか」を呼び出し元まで遡らないと安全性を判定できない。タグ付きテンプレートなら `${}` が構文上必ずパラメータになるので、その関数だけ読めば判定が閉じる。

トレードオフは表現力で、識別子や `ORDER BY` の列名を動的に変えたいくなった瞬間に `Prisma.raw` が必要になり、`*Unsafe` と同じ検証コストが戻ってきます。今回の3クエリは値しか動かないので失うものがなかった、という判断です。

根拠

"Wherever possible you should use the `$queryRaw` method instead. When used correctly `$queryRaw` method is significantly safer"

— [Prisma — Raw queries](#)

追い質問と返し

「じゃあ `$queryRaw` なら絶対安全なの？」

いいえ。`Prisma.raw()` や `Prisma.sql` で文字列を混ぜれば注入できます。公式も "it is still possible to introduce SQL injection if these methods are used in certain ways" と書いています。防げるのは「補間値が SQL になる」経路であって「SQL 自体を組み立てる」経路ではない。

「`::vector` キャストがあるのにタグ付きテンプレートで書けたの？」

書けました。実際に pgvector コンテナを立てて確認しています。Prisma は SQL 本文を ...

`(pe.embedding <=> $1::vector)::float8 ... ORDER BY pe.embedding <=> $2::vector LIMIT $3` に変換し、Parse → Bind → Execute で投げます。 `::vector` は「パラメータを vector にキャストする」という構文木の一部で、値側ではなく SQL 側に属するので補間と衝突しません。同じベクタを2回補間しているので Bind は `$1` と `$2` に同じ値を2回送ります。

「ベクタを text で送るのは無駄では？」

そのとおりで、768次元だと約 8~10KB の文字列になります。バイナリ形式 (`4*768+8 = 3,080` バイト) に比べて3倍前後。pgvector の公式 Node クライアントは型登録でバイナリ送信できますが、Prisma の raw API は独自プロトコルなので同じ手が使えない。Prisma を選んだ結果として受け入れているコストです。

Q2. HNSW インデックスは本当に効いていますか？ `JOIN post WHERE isPublic = true` がありますよね。

該当箇所: `src/embeddings/embeddings.service.ts:110-119` / `prisma/migrations/20260501000000_add_pgvector_post_embedding/migration.sql:20-22`

回答

効きますが、制約付きで効くという理解です。

pgvector 公式が明言するとおり、近似インデックスでは「filtering is applied **after** the index is scanned」なので、この JOIN は post-filter です。HNSW スキャンが返す候補は `hnsw.ef_search` (既定 40) に制限され、そこから `isPublic = false` の行が落ちると LIMIT に満たない件数しか返らない可能性がある。

ただしこの設計では `post_embedding` にそもそも非公開投稿の行を入れない方針を取っています (`posts.service.ts:110-129` の `syncEmbedding` が非公開なら `deleteForPost` を呼ぶ / 既存行は `scripts/purge-private-embeddings.ts` で削除)。つまり JOIN の選択率は設計上ほぼ 100% で、post-filter による目減りは起きにくい。JOIN は「埋め込み削除が失敗した場合」の二重防御として残しています。

`ORDER BY` の演算子 `<=>` はインデックスの opclass `vector_cosine_ops` と一致しているので、`ORDER BY + LIMIT` のパターンで index scan が選ばれます。

根拠

"With approximate indexes, filtering is applied after the index is scanned. If a condition matches 10% of rows, with HNSW and the default `hnsw.ef_search` of 40, only 4 rows will match on average"

— [pgvector](#) — Filtering

追い質問と返し

「埋め込み削除が失敗したら？」

`deleteForPost` は `embeddings.service.ts:83-87` で例外を握り潰して `warn` ログを出すので、失敗すると非公開投稿の行が残り、`post-filter` が実際に働いて結果が目減りします。正しくは (a) 削除を失敗させない（投稿更新と同トランザクション、または `outbox` / リトライ）か、(b) `post_embedding` に `isPublic` を非正規化して `partial index`（`CREATE INDEX ... WHERE "isPublic"`）にする。公式も低カーディナリティのフィルタには `partial index` を推奨しています。◆ 未対応。

「`hnsw.iterative_scan`（pgvector 0.8.0+）は使わないの？」

`iterative_scan` は「`post-filter` で目減りする」問題の薬です。この設計は埋め込みテーブルに公開投稿しか入れないことで目減りの原因そのものを消しているので、効く相手がいない。導入すれば `SET LOCAL` の運用コストと `hnsw.max_scan_tuples`（既定 20,000）のチューニングが増えるだけです。加えて RAG の上位 `k` を取る用途では距離順が厳密な `strict_order` 一択になるので速度メリットも小さい。

将来「著者で絞る」「タグで絞る」のような選択率の低いフィルタをベクトル検索に足すなら、そのとき `iterative_scan` か `partial index` を検討します。

Q3. `MAX_LIMIT` は 40 ですが、`hnsw.ef_search` はいくつですか？

該当箇所: `src/embeddings/embeddings.service.ts:17-20`, 95

回答

設定していないので既定の 40 です。そして元は `MAX_LIMIT = 50` で、これは契約と実装が食い違っていました。

pgvector の HNSW スキャンは動的候補リストのサイズを超える行を返せないため、`limit` が 41~50 のときフィルタが一切効かなくても最大 40 行しか返りません。呼び出し元が渡す `limit` は実際には 5 なので顕在化していませんでしたが、API の契約としては壊れていた。今回 40 に揃えました。

この修正の弱点を3つ、自分から言います。

- 40 が GUC 既定値の二重管理になっている — `MAX_LIMIT = 40` は `hnsw.ef_search` の既定値のミラーで、その依存がコードに書かれていない。DBA がパラメータグループで 100 に上げてても追従しないし、誰かが 50 に戻しても CI は通ります。
- 余裕がゼロ — `ef_search = 40` / `LIMIT = 40` は、JOIN が1行でも落とせば 40 行に届かない。実用上は `LIMIT ≤ ef_search / 2` が定石で、安全側なら 20 にすべきところです。
- `recall` のチューニングポイントを閉じた — `ef_search` は上限であると同時に探索の質そのもの。本来 `limit`（API 都合）と `ef_search`（`recall` 都合）は独立に決めるべきなのに、1つの定数に潰しました。

本質的には `SET LOCAL hnsw.ef_search` でアプリ側が GUC を所有するのが正しい。40 にしたのは既存の API 契約を壊さない範囲での最小修正です。

根拠

"Results are limited by the size of the dynamic candidate list (`hnsw.ef_search`), which is 40 by default"

— [pgvector README](#)

追い質問と返し

「じゃあ `SET LOCAL` にすればよかったのでは？」

そちらも安くはありません。(a) トランザクションスコープなので明示トランザクションが必須になりラウンドトリップが +1、(b) GUC 名も値もパラメータ化できない（公式「Variables cannot be used for identifiers ... or for SQL keywords」）ので `Prisma.raw` が1箇所復活する — せっかく `*Unsafe` を消した意味が一部後退する、(c) `LOCAL` を忘れて `SET` にするとプールされたコネクションに設定が residue として残り他リクエストに漏れる。

折衷案は接続文字列に `?options=-c hnsw.ef_search=100` を付けてプール全体に固定することで、トランザクション不要・`Prisma.raw` 不要・漏れの概念なし、代わりにクエリ単位の調整ができません。

Q4. なぜ 768 次元なんですか？ Gemini の埋め込みは 3072 次元ですよ。

該当箇所: `prisma/schema.prisma:66` / `src/embeddings/embeddings.service.ts:10` / `nokoro-ai/app/services/gemini_service.py:63-73`

回答

pgvector のインデックス上限が 2,000 次元だからです。

`gemini-embedding-001` の既定出力は 3,072 次元ですが、pgvector の HNSW / IVFFlat がインデックスできるのは 2,000 次元まで。そのまま入れるとインデックスが張れず全件シーケンシャルスキャンになります。

そこで Gemini API の `output_dimensionality=768` で MRL (Matryoshka Representation Learning) 縮約をかけ、DB スキーマ (`vector(768)`) と一致させました。768 を選んだのは 2,000 未満で公式が推奨している値 (768 / 1536 / 3072) のうち最小だから。ストレージは $4 \times 768 + 8 = 3,080$ バイト/行 (3,072 次元なら 12,296 バイト)。

根拠

"up to 16,000 dimensions" / インデックス可能なのは "up to 2,000 dimensions" / "Each vector takes $4 * \text{dimensions} + 8$ bytes of storage"

— [pgvector README](#)

"We recommend using 768, 1536, or 3072" / "trained using the Matryoshka Representation Learning (MRL) technique"

— [Gemini API — Embeddings](#)

追い質問と返し

「公式は『3072 以外の次元は手動で正規化が必要』と書いていますよね。正規化していませんよね？」

していません。実害が出ていないのは検索が `<=>` (cosine distance) で、インデックスも `vector_cosine_ops` だからです。cosine はノルムに対してスケール不変なので順位は正規化の有無で変わりません。

ただしこれは「たまたま守られている」状態で、将来 `<->` (L2) や `<#>` (内積) に変えた瞬間に壊れます。正しくは `embed()` の戻り値で L2 正規化してから保存すべきで、◆ 直すべき箇所として認識しています。

「次元を変えたくなかったら？」

`ALTER TABLE ... ALTER COLUMN embedding TYPE vector(N)` は既存データと非互換なので実質全行再生成です。運用としては新カラム追加 → バックフィル → 切り替えの3段。2,000 を超えたいなら `halfvec` (fp16、4,000 次元までインデックス可) にキャストして `halfvec_cosine_ops` で張る、が公式の回避策です。

「次元数は何箇所に散っていますか？」

3箇所 (backend の `EMBEDDING_DIM`、AI サービスの `settings.embedding_dim`、DB スキーマ) です。ズレると `$executeRaw` が実行時に `expected 768 dimensions` で落ちる。backend 側は AI サービスの応答長を検証して DB 到達前に弾いていますが、◆ 3箇所の同期はコメントによる規約でしか担保されていないのが弱点です。

Q5. `ThrottlerGuard` をグローバルに置いているのに、チャットだけコントローラ側に書いているのはなぜ？

該当箇所: `src/app.module.ts:26-45` / `src/chat/chat.controller.ts:17-22` / `src/common/user-throttler.guard.ts`

回答

グローバルガードでは原理的に `req.user` が読めないからです。

NestJS の `GuardsContextCreator` は `globalGuards.concat(scopedGuards)` を返し、`GuardsConsumer` がその配列を先頭から逐次実行します（実装で確認）。つまり `APP_GUARD` は必ず `@UseGuards` より先に走り、`JwtAuthGuard` がまだ実行されていない。ユーザー単位で数えるには認証済みであることが前提なので、スロットラーを認証ガードの後ろに置く必要があり、それを表現できるのは `@UseGuards(JwtAuthGuard, UserThrottlerGuard)` という順序付きリストだけです。

ここで一度実装を間違えて、テストで捕まえました。最初は「`shouldSkip` で `req.user` を見て分岐する」実装にしたのですが、同じ理由で `shouldSkip` の時点でも `req.user` は常に `undefined`。結果、認証済みでもスキップされず IP 単位の 20/分が復活し、同一 IP の別ユーザーが巻き添えて 429 になる元の症状に戻っていました。e2e が落ちて気づきました。

最終形は名前付き throttler です。

throttler	上限	単位	評価するガード
default	100/分	IP	グローバル（全経路）
user	20/分（chat で上書き）	ユーザー	UserThrottlerGuard（認証後）

`user` には `skipIf: ctx => isTestEnv() || trackedUserId(ctx) === undefined` を付けているので、グローバルガードでは必ずスキップされ、認証後の `UserThrottlerGuard` だけが評価します。

根拠

- 実装で確認: `@nestjs/core/guards/guards-context-creator.js:67` の `globalGuards.concat(scopedGuards)`
- 公式: 「Guard execution starts with global guards, then proceeds to controller guards, and finally to route guards」 — [NestJS — Request lifecycle](#)
- 実測（e2e）: ユーザーA が 21 回目で 429 → 直後に同一 IP の別ユーザーB は通過（`test/throttle.e2e-spec.ts`）

追い質問と返し

「なぜ IP 単位とユーザー単位を混在させるの？」

守る対象が違うからです。ログイン・登録は未認証なので原理的に IP しか鍵にできない。AI チャットは Gemini の従量課金が走るので、IP で数えると1アカウントが IP をローテートして回避できる。だから経路ごとに単位を選び分けています。

「ALB 配下で `req.ip` は取れてる？」

`main.ts:22` の `app.set('trust proxy', 1)` で取れます。Express は `X-Forwarded-For` を右から左に見て、ALB が append した右端の値を採用します。ALB の既定は append（末尾にクライアント IP を追記）で、ECS の SG は ALB の SG からのみ inbound を許可しているので、攻撃者が XFF を偽装しても偽装値は左側に積まれるだけで `req.ip` は動きません。

`trust proxy: true` にしなかったのは、`true` だと左端を無条件に信頼して詐称できるからです。ALB の前に CloudFront を挟んだらここを 2 に上げる必要があります。

「ECS を複数タスクにしたら？」

◆ 破綻します。 `ThrottlerModule` の既定ストレージはプロセス内 Map なので、N タスクで実効上限が N 倍になり、再起動でカウンタも消えます。スケールアウト前に Redis ストレージ (ElastiCache) へ寄せる必要があります。現状 1 タスク運用なので顕在化していない、というだけです。

Q6. JWT を localStorage に入れていますよね。OWASP は httpOnly Cookie 推奨だし、`SECURITY.md` に自分で「リスクあり」と書いてありますよね。

該当箇所: `nokoraa-frontend/src/utils/secureAuth.ts` / `src/providers/AuthProvider.tsx` / `nokoraa-frontend/SECURITY.md`

回答

◆ 直していません。意図的に残した未対応項目です。

理由は「Cookie に移すことが単独の変更にならない」から。Cookie に載せた瞬間ブラウザが自動送信するようになるので、CSRF 対策がセットで必須になり、`SameSite` の設計、オリジン検証、`enableCors({credentials:true})` の厳格化、そして OAuth コールバックのトークン受け渡しの作り直しまで一続きになります。

現状は `Authorization: Bearer` を明示的に付ける方式なので、ブラウザが自動送信しない=CSRF 面ではゼロコストという利点はあります。ただしその代償が XSS 耐性で、OWASP の言う「1個の XSS で全部盗まれる」状態であることは認めます。

正しくは 短命アクセストークン (メモリ保持) + httpOnly の refresh Cookie + CSRF トークンの構成です。

根拠

"Do not store session identifiers in local storage as the data is always accessible by JavaScript." / "A single Cross Site Scripting can be used to steal all the data in these objects."

— [OWASP HTML5 Security Cheat Sheet](#)

追い質問と返し

「じゃあ XSS 対策は？」

React の既定エスケープに加えて、今回 Google Maps の InfoWindow を HTML 文字列から DOM 構築に直しました (`GoogleMap.tsx`)。ここは React の外に出る唯一の HTML sink で、投稿タイトルに `<img src=x onerror=...>` を入れれば実行できていた箇所です。

◆ **ただし CSP は入れていません。** Google Maps と MUI(emotion) のインラインスタイルに依存していて nonce 化が別作業なので、まず HSTS / nosniff / X-Frame-Options / Referrer-Policy / Permissions-Policy から入れました。CSP 無し + localStorage は「XSS → 即アカウント乗っ取り」なので、優先度は CSP の方が高いと今は判断しています。

「sessionStorage や in-memory じゃダメだったの？」

in-memory はリロードで落ちるので refresh トークン機構が前提。sessionStorage は OWASP 的には localStorage と同じ扱い (JS からアクセス可能) なので、XSS 面の改善にならずタブ間で切れる副作用だけ増えます。だから「中間的な小改善」はせず、未対応として明示する選択をしました。

Q7. `jwt.strategy.validate` が DB を引いていません。退会後やパスワード変更後のトークンは何時間有効ですか？

該当箇所: `src/auth/jwt.strategy.ts:21-23` / `src/auth/auth.module.ts:18`

回答

24 時間有効です。失効機構はありません。 `validate` は payload をそのまま写すだけで DB 照会をしないので、退会・パスワード変更・ログアウト後も署名と `exp` が有効な限り通ります。

これは JWT を stateless セッションとして使った場合の構造的な帰結で、OWASP も "A JWT is often suggested for 'stateless' user sessions. However, this usage is frowned upon" と明言しています。

選択肢は3つ: (a) `validate` で毎回 user を引いて `tokenVersion` と突き合わせる、(b) `jti` ベースの deny list を Redis に置く、(c) アクセストークンを 15 分に縮めて refresh 側で失効させる。◆ **どれも入れていません。**

根拠: [OWASP JWT Cheat Sheet](#)

追い質問と返し

「3つのうちどれを選ぶ？」

(a) の `tokenVersion` です。deny list は「トークン1本ごとに1エントリ」で失効対象を列挙する形なので、「パスワード変更でそのユーザーの全トークンを落としたい」という要件と粒度が合いません。

`user.tokenVersion` を payload に入れて `validate` で突き合わせれば、1カラムの UPDATE で全端末を落とせます。DB 往復が気になるなら user を短 TTL でキャッシュし、bump 時に invalidate する。deny list は「特定端末だけログアウト」が要件に入った時点で追加します。

「なぜ `expiresIn: '1d'` なの？」

refresh 機構が無いので、短くするとユーザーが毎回再ログインになるからです。つまり「1d」は失効機構が無いことの裏返しで、独立した判断ではありません。refresh を入れた時点でアクセストークンは 15 分に縮めます。

「フロントの自動ログアウトタイマーは？」

クライアント側だけの UX 機構で、サーバから見たトークンの有効性には一切影響しません。攻撃者がトークンを抜いて curl で使う場合には何の意味もないので、防御として数えていません。

Q8. App Router を使っているのに、ほぼ全ページが `'use client'` ですね。App Router を使う意味ありますか？

該当箇所: `nokoroa-frontend/src/app/` 配下 29 ファイル中 27 が `'use client'`

回答

RSC の恩恵はほぼ受けられていません。理由は認証設計にあって、JWT を `localStorage` に置いているためサーバ側にトークンが渡らず、データ取得を Server Component で行えない構造になっています。結果「認証状態を読む → データを取る」が全部クライアントに寄り、`useAuth()` を参照する page が芋づる式に `'use client'` になりました。

ただし App Router を選んだこと自体は無駄ではなく、**Parallel Routes + Intercepting Routes** (`app/@dialog/(.)login/page.tsx`) でログインモダルの URL 同期を実装しています。これは Pages Router では自前ルーティングが必要になる部分です。

正しくは、トークンを `httpOnly Cookie` に移して page を Server Component にし、`'use client'` は `BookmarkButton` / `ChatPanel` のような葉に押し下げるべきです。

根拠

"add `'use client'` to specific interactive components instead of marking large parts of your UI as Client Components"

— [Next.js — Server and Client Components](#)

追い質問と返し

「じゃあ Pages Router でよかったのでは？」

機能要件だけ見ればそうです。ただ `layout.tsx` は現状も Server Component なので、**Cookie 認証に切り替えれば page 単位で段階的に RSC 化**できる。Pages Router だとこの移行パスがありません。Q6 と Q8 は同じ根 (`localStorage`) から出ている、というのが整理です。

第2部 — よく聞かれる

Q9. `PostWithRelations` を手書きして `as` でキャストしていますよね。Prisma は型を生成しますよね？

該当箇所: `src/posts/posts.service.ts:49-83` と `as` キャスト6箇所 / `src/favorites/favorites.service.ts` に2箇所

回答

◆ 正当化できません、負債です。

`postInclude` という定数から型を導出できるのに手書きしているので、スキーマを変えても TypeScript は何も言わず、`as` が型エラーを黙らせます。実際 `favorites.service.ts` 側の `PostWithRelations` には `location.createdAt` が無く、`posts.service.ts` 側にはある — 同名の型が2つ、しかも中身が違う状態です。

正しくは公式推奨の `satisfies` + `GetPayload` :

```
const postArgs = { include: postInclude } satisfies Prisma.PostDefaultArgs;
type PostWithRelations = Prisma.PostGetPayload<typeof postArgs>;
```

これで `postInclude` の変更が型に自動追従し、`as` は全部不要になります（`findMany` の戻り値がそのまま代入可能になるため）。

根拠

"Reduced maintenance burden and improved type safety when the schema changes"
— [Prisma — Operating against partial structures of model types](#)

追い質問と返し

「`as` が具体的に何を隠していますか？」

`publicAuthorSelect` から `avatar` を外した瞬間、`formatPost` 内の `post.author.avatar` は型上は生きたまま実行時 `undefined` になります。`GetPayload` ならコンパイルエラー。もう一例、`Post` に新カラムを足すと手書き `interface` は古いまま `as` で通り、レスポンスに載っているのに型定義に無い乖離が起きます。

Q10. `req.user` の形が `posts` では `{ id }`、他は `{ userId }` で揃っていませんね。

該当箇所: `src/auth/jwt.strategy.ts:22` / 各コントローラ

回答

◆ 設計判断ではなく、歴史的経緯を `strategy` 側で吸収した技術的負債です。 `posts` が `id` を、後から足したモジュールが `userId` を前提に書かれ、どちらかに寄せると呼び出し側を全部直す必要があったので、`validate()` が両方を返して両立させています。

ここに関連して、型が完全に嘘になっている箇所がありました（今回修正済み）。

`interface AuthenticatedRequest extends Request` の `Request` が `@nestjs/common` からの `import` で、これはパラメータデコレータという「値」であって型ではありません。結果 `extends Request` は `Fetch API` の `Request` に解決されていました。

実際に `tsc` で確認した結果:

アクセス	結果
<code>r.params</code> (Express なら存在)	<code>Property 'params' does not exist</code>
<code>r.bodyUsed</code> (Fetch API なら存在)	通る

つまりこの `interface` は「Express の `req` に `user` を足した型」ではなく「fetch の `Request` に `user` を足した別物」で、`user` 以外は全部嘘でした。 `common/authenticated-request.ts` に集約し、`express` の `Request` を明示的に `import` する形に修正しています。

根拠: [NestJS — Custom decorators](#) (`createParamDecorator` による `@User()` パターン)

追い質問と返し

「`AuthenticatedRequest & { user?: ... }` で optional にしていた箇所は？」

これも無意味でした。TypeScript の交差型ではプロパティが optional になるのは全構成要素で optional な時だけなので、`{user: T} & {user?: T}` の `user` は `required` のままです。実際に守っていたのは実行時の `?.` だけ。専用の `OptionallyAuthenticatedRequest` 型に置き換えました。

「なぜ気づけなかった？」

`tsconfig.json` が `strict: true` を書きつつ `strictNullChecks: false` / `noImplicitAny: false` で明示的に打ち消しているためです。◆ 「strict を有効にしている」という外観だけが残っている状態で、これも未対応の負債です。

Q11. `posts.service.ts` が 710 行あります。大きすぎませんか？

該当箇所: `src/posts/posts.service.ts`

回答

大きすぎます。混在している関心事は最低4つ:

- 投稿の CRUD と所有権チェック
- `location` / `tag` の upsert (本来 `LocationsService` / `TagsService` に分けるべき別集約)
- 3種類の検索 (Prisma クエリビルダ / Haversine 生 SQL / 集計)
- 埋め込みの副作用同期

`searchByLocation` の Haversine 式が同一ファイル内に3回重複しているのは `$queryRaw` のタグ付きテンプレートを選んだコストです。 `$queryRawUnsafe` なら文字列連結で WHERE 句を1箇所にできますが、それは SQL インジェクションの窓口を開けることと引き換え。安全性を取って重複を許容した、というのが判断です。

追い質問と返し

「重複を消す方法は本当に無い？」

あります。 `Prisma.sql` ヘルパで WHERE 句を `const whereClause = Prisma.sql\ ...\`` として切り出し、両方のクエリに `${whereClause}`` で埋め込めば、パラメータ化を保ったまま1箇所にできます。やっていないのは単に手を入れていないからで、技術的な障壁はありません。

「分割するとしたら？」

`PostsService` (CRUD + 所有権) / `PostSearchService` (3種の検索) / `LocationsService` + `TagsService` (upsert)。 `syncEmbedding` は直接呼び出しをやめて `EventEmitter2` で `post.created` / `post.updated` を発行し `EmbeddingsListener` が購読する形にすれば、 `PostsModule` → `EmbeddingsModule` の依存が1本消えます。

「Haversine を毎行計算しているのは遅くない？」

遅いです。 WHERE 節の距離式は列に対する関数なので B-tree インデックスが効かず、結合結果に対して全件で `acos` / `cos` / `sin` を評価します。本来は bounding box で事前絞り込みするか、PostGIS の `geography` + GiST (`ST_DWithin`) に置き換えるのが筋。 `pgvector` を既に入れているので拡張を足す抵抗は小さいはずです。

Q12. チャットのストリーミングで `@Res()` を使っていますね。 `@Sse()` があるのに。

該当箇所: `src/chat/chat.controller.ts` / `src/chat/chat.service.ts`

回答

`@Sse()` は「`Observable<MessageEvent>` を返すと Nest が `data:` 形式にシリアル化する」仕組みで、自前でイベントを組み立てたい場合に合いません。ここでは上流の Python サービスが既に SSE フレームを返すので、それをバイト列でパススルーしたい。`@Sse()` に載せるには一度パース → 再シリアル化が必要で、無駄な往復と壊れやすい処理が増えます。

ただし代償は大きく、公式が明記するとおり「you lose compatibility with Nest features that rely on standard response handling」で、例外フィルタも実質効きません。実際このコードは `res.status(502).json(...)` をサービス層で手書きしており、サービスがトランスポート層の知識を持っています。

根拠: [NestJS — Controllers \(library-specific approach\)](#) / [NestJS — SSE](#)

追い質問と返し

「具体的にどんなバグが起き得ますか？」

2つあります。(1) `reader.read()` がストリーム途中で throw すると `finally { res.end() }` が走って接続を閉じた後に例外が伝播し、Nest の例外レイヤが終了済みの `res` に書こうとして `ERR_HTTP_HEADERS_SENT` になる。(2) ◆ クライアントが切断しても上流 fetch の `reader` は読み続けるので、`@Sse()` が自動でやってくれる `unsubscribe` 相当が無く、上流への課金が続きます。最低限 `res.on('close', () => reader.cancel())` を足すべきで、未対応です。

Q13. SSE のパースで `data:` 行を集めて改行で join していますね。普通は1行目だけでは？

該当箇所: `nokoraa-frontend/src/components/chat/ChatPanel.tsx:246-255` / `nokoraa-ai/app/routers/chat.py`

回答

SSE の仕様がそうになっているからです。WHATWG のパーサは `data` フィールドを見つけるたびに「Append the field value to the data buffer, then append a single LF character」と定義していて、1イベントが複数の `data:` 行を持つのは正常なケースです。

Gemini の生成テキストには改行が入るので、送信側で `payload.split("\n")` して各行に `data:` を付けています。受信側で1行目だけ読むと2行目以降が丸ごと欠落する — これは実際にバグとして存在し、今回修正しました。`.slice(6)` で `"data: "` を落としているのも仕様の「If value starts with a SPACE character, remove it」に対応しています。

根拠: [WHATWG HTML — Server-sent events](#)

追い質問と返し

「EventSource を使えばパースは不要では？」

そのとおりですが、EventSource は GET しか送れず Authorization ヘッダも付けられないので、POST + fetch + 手動パースを選んでいきます。

「id: や retry: 、コメント行は？」

◆ 未対応です。今は自分で送るイベントだけを相手にしていて、backend も生バイトをそのまま流すので data: 以外は発生しません。data: 以外を無視する現在の filter で壊れはしませんが、retry: による再接続を使いたくなったら EventSource 系に寄せます。

Q14. なぜ AI を NestJS 内で完結させず Python の別サービスに？ しかも ECS のサイドカーですよ。

該当箇所: [nokoroa-ai/](#) / [terraform/modules/ecs/main.tf](#)

回答

技術的な理由は3つ。

- SDK の先行度 — google-genai の Python SDK が Node 版より先行していて、output_dimensionality による MRL 縮約や Google Search Grounding が先に来るのが Python 側でした。
- 信頼境界の分離 — AI 呼び出しは従量課金を伴う外向きの境界なので、プロセスを分けて INTERNAL_API_KEY の検証をルーター単位で必須化し、backend を通らない呼び出しを構造的に拒否できるようにした。
- 運用単位を増やさない — 別 ECS サービスではなく同一タスク定義のサイドカーにして、Fargate の awsvpc モードで「同一タスクのコンテナは localhost で通信できる」性質を使い、portMappings を一切付けずに ALB・SG・サービスディスカバリを増やさずに済ませています。

トレードオフは、デプロイ単位が backend と一体化することと、スケール粒度を分けられないことです。

根拠: [AWS — Fargate task networking](#)

追い質問と返し

「portMappings が無いのに localhost:8000 で繋がるのは？」

portMappings は awsvpc では「タスク ENI の外に晒すか」の宣言で、同一タスク内のコンテナは同じネットワーク名前空間を共有するのでループバックでは常に到達可能です。書かないことが「外部に晒さない」という意図の表明になります。

「AI サービスのイメージはどうやってデプロイしている？」

◆ これは穴です。 `deploy.yml` に AI のビルド・push ジョブが無く、手動運用になっています。さらに `ai_image` が未指定だとフォールバックが素の `python:3.12-slim` で、これは `uvicorn` を起動しないため必ずヘルスチェックに落ち、`backend` が `dependsOn: HEALTHY` で待っているのでタスク全体が起動しません。「設定漏れ＝即障害」という危険なデフォルトです。

Q15. デプロイに `prisma migrate deploy` がありません。本番のマイグレーションは？

該当箇所: `.github/workflows/deploy.yml` (`migrate` の出現 0 件) / `.github/workflows/ci.yml:96`

回答

◆ 走りません。これが現状の欠落です。

`migrate deploy` は CI の ephemeral な Postgres に対して e2e の前準備として実行されているだけで、本番 RDS には誰も適用していない。デプロイは ECR push → タスク定義更新 → サービス再デプロイだけなので、スキーマ変更を含むリリースは新コードが旧スキーマに対して動く状態になります。

正しくは ECS デプロイの前に `migrate` ステップを挟む。RDS が private subnet なので GitHub Runner から直接は届かず、`aws ecs run-task` でワンショットタスクを流して終了コードを待つのが現実的です。

根拠: [Prisma — Development and production workflows](#)

追い質問と返し

「コンテナ起動時に流せばよいのでは？ Postgres なら advisory lock で待つし。」

壊れはしません (Prisma 公式も「one of them waits for the other」と書いています)。ただ待っている間コンテナが起動完了せず、ヘルスチェックの `startPeriod = 60` を超えるとタスクが kill されてデプロイがロールバックします。実用上は `run-task` 分離のほうが安全です。

「`CREATE EXTENSION IF NOT EXISTS vector` は RDS で通りますか？」

ここも要注意です。PostgreSQL 公式は「For many extensions this means superuser privileges are needed」としていて、RDS には真の superuser が無く `rds_superuser` ロールになります。通常アプリケーションユーザーには渡さない権限なので、◆ この1行だけは Prisma のマイグレーション履歴から外し、インフラ側で先に張っておくのが正しい分離です。現状はマイグレーションに埋まっているので本番適用時に権限で止まる可能性があります。

「本番の PostgreSQL バージョンは？」

terraform/modules/rds/variables.tf の engine_version 既定値が 15.15 で、envs/prod/main.tf では上書きしていません。一方ローカルと CI は pgvector/pgvector:pg16 。 ◆ dev/prod parity のギャップです。

HNSW 自体は動きます (RDS PG 15.4-R3 以降で pgvector 0.5.1)。ただし hnsw.iterative_scan は pgvector 0.8.0 = PG 16.5 以降が必要なので、ローカルでは使えて本番では使えないという差があります。

第3部 — 掘られたら答える

Q16. getOrCreateTags が Promise.all の中で findUnique → create していますね。

該当箇所: src/posts/posts.service.ts:158-173

回答

◆ N+1 です。タグ N 個で最大 2N クエリ、しかも Promise.all なので N 本の接続を同時に掴みます。さらに競合に無防備で、同じタグ名の投稿が同時に2件作られると両方の findUnique が空を返し、両方が create して片方が一意制約で P2002 になります (Prisma 公式 issue #3242 と同型)。

正しくは createMany({ data, skipDuplicates: true }) を1回投げてから findMany({ where: { name: { in: tags } } }) で1回引く (計2クエリ)。skipDuplicates は PostgreSQL では ON CONFLICT DO NOTHING なので競合が例外にならないのが利点です。

根拠: [Prisma — CRUD](#)

Q17. タグ更新が deleteMany → createMany → update で、\$transaction の外ですね。

該当箇所: src/posts/posts.service.ts:370-391

回答

◆ 意図的ではなく、直すべき箇所です。deleteMany が成功して createMany が失敗すると、その投稿のタグが全消失した状態でコミット済みになります (Prisma は個々のクエリを暗黙の単一文トランザクションとして実行するため)。

正しくは interactive transaction にまとめる。ただし getOrCreateTags は外部往復を含むのでトランザクション外に出し、postTag の入れ替えだけの中に入れるのが妥当です (既定 timeout は 5,000ms なので外部 I/O を中に入れない原則)。

根拠: [Prisma — Transactions](#)

Q18. COUNT(*) を Number() で包んでいるのに、p.id は包んでいませんね。

該当箇所: `src/posts/posts.service.ts` の COUNT クエリ群

回答

PostgreSQL の `COUNT(*)` は `int8` (`bigint`) を返し、Prisma の raw クエリ型マッピングでは `BigInt` になります。`BigInt` は `JSON.stringify` でシリアライズできず `TypeError` で 500 になるので、レスポンスに載せる前に `Number()` へ落としています。一方 `p.id` は `SERIAL = int4` なので Prisma が最初から `Number` で返す。だから包む必要がない。

型注釈を `<{ count: bigint }[]>` と書いているのはこの事実を明示するためですが、`$queryRaw<T>` の型注釈は実行時に検証されないので、これは「ドキュメント」であって保証ではありません。

根拠: [Prisma — Raw queries](#) (型マッピング表 / "the raw data might not always match the suggested TypeScript type")

Q19. 非公開投稿に他人がアクセスしたとき 403 ではなく 404 ですね。

該当箇所: `src/posts/posts.service.ts:320-324` (404) / 同 `:342-344` (update は 403)

回答

意図的に 404 です。403 を返すと「その ID の投稿は存在する（が見えない）」という情報を渡してしまい、ID を総当たりすれば非公開投稿の分布が観測できます。OWASP Authorization Cheat Sheet の「認可エラーから情報を漏らさない」に沿って存在秘匿を優先しました。

一方 `update` / `remove` は 403 のままです。「自分の投稿を編集しようとして 404」だと原因切り分けが不可能になるため、そもそも認証必須の経路なので 403 が漏らす情報は限定的。読み取りは匿名でも叩けるので秘匿優先、書き込みは認証済み前提で診断性優先、という線引きです。

根拠: [OWASP Authorization Cheat Sheet](#)

追い質問と返し

「404 にしても漏れる経路は？」

◆ 応答時間です。存在しない ID は `findUnique` が空で即返り、存在する非公開 ID は JOIN と `bookmark.count` を実行した後に 404 になります。理屈の上ではタイミング差が出る。潰すなら可視性判定を先にやって、見えないと決まった時点で以降のクエリを打たない形に組み替えます。未対応です。

Q20. OptionalJwtAuthGuard が err を捨てていますね。改竄トークンでも 200 ですよ。

該当箇所: `src/auth/guards/optional-jwt-auth.guard.ts`

回答

安全だと判断しています。根拠は passport-jwt の制御フローで、署名検証の失敗時は `self.fail(jwt_err)` を呼びます。passport の `fail` は (`err=null, user=false`) でコールバックするため、改竄・期限切れトークンは `handleRequest` に `user = false` で届き、`undefined` が返ります。結果 `req.user === undefined` となり、`findOne(id, undefined)` で 404、`findById(id, undefined)` で `isOwner=false`。

つまり「偽トークン = 匿名」であり、偽トークンで他人になれる経路は存在しません (fail-closed)。

代償は UX と診断性で、期限切れトークンを持ったオーナーが自分の非公開投稿を開くと 401 ではなく 404 が返り、フロントは「再ログインが必要」を判別できません。

根拠: 実装で確認 (`passport-jwt/lib/strategy.js` の `fail` 呼び出し、
`@nestjs/passport/dist/auth.guard.js` の `request[property] = user; return true;`)

追質問と返し

「err が非 null になるケースは本当に無い？」

あります。passport-jwt が `self.error(ex)` を呼ぶのは `verify` コールバック (= `validate()`) が例外を投げた時です。現状の `validate()` は純粋な写像なので投げる余地がありませんが、◆ 将来 `validate()` で DB を引くようにしたら、DB 障害という 500 相当の事象が「匿名アクセス」に化けます。

正しくは `if (err) throw err; return user ?? undefined;` として「認証失敗は通す／サーバ側エラーは通さない」を区別すべきで、今は `validate()` が throw しない前提に依存した実装です。その前提はコードに書かれていません。

Q21. RAG で他ユーザーの投稿がプロンプトに入りますよね。プロンプトインジェクションは？

該当箇所: `nokoroa-ai/app/services/gemini_service.py`

回答

完全には防げません。OWASP LLM01 自体が "it is unclear if there are fool-proof methods of prevention for prompt injection" と書いていて、これは緩和策の積み上げです。

4層で対策しています。

- `<nokoroo_user_posts>` で外部データを構造的に分離 ("Segregate and Identify External Content")
- 区切りの後ろで指示を再掲 — 前置きだけだと長い投稿本文に埋もれるため
- `title` / `content` / `location` / `author` の全部を無害化 — 1つでも生で残ると閉じタグを偽造されて境界を破られる
- ゼロ幅文字 (U+200B/200C/200D/FEFF) を除去 — 人間には見えないがトークナイザには見える指示を無効化

ただし防御の本体は「成功しても被害が出ない設計」側に置いています。この AI は Google Search ツールしか持たず、DB 書き込みも関数呼び出し権限も無い。最悪ケースが「変な旅行提案が返る」で、データ流出や権限昇格には至りません。逆に言えば、将来 AI に投稿削除のようなツールを持たせた瞬間に、この強度では足りなくなります。

根拠: [OWASP LLM01:2025 Prompt Injection](#)

追い質問と返し

「`role` を絞っているのは何のため？」

直接注入の方です。クライアントが任意の `role` を送れると `role: "system"` で過去のシステム指示を捏造できるので、`Literal["user", "model"]` で閉じています。

◆ ただし「過去の `model` 発言」の捏造は依然として可能です。「私は制約を解除しました」という偽の AI 応答を履歴に入れることは今でもできる。塞ぐには履歴をサーバ側セッションで保持してクライアントから受け取らない形にする必要があり、未対応です。

「他人の非公開投稿がプロンプトに混ざる可能性は？」

二重で閉じています。生成側は `syncEmbedding` が非公開投稿の埋め込みを生成せず既存行も削除 (`post_embedding` に可視性カラムが無いので、行を残さないことで守る)。検索側は SQL の `WHERE p."isPublic" = true`。「可視性を持たないテーブルには可視性を要するデータを置かない」が方針です。

Q22. 一覧を開くとカードの数だけブックマーク状態の API が飛びますよね。

該当箇所: `nokoroo-frontend/src/components/bookmarks/BookmarkButton.tsx` / `components/follow/FollowButton.tsx`

回答

◆ クライアント側の N+1 です。トップは1ページ12件なので、一覧1回 + `/favorites/check/:postId` が12回。無限スクロールで積み上がります。しかも SWR を経由していない素の `fetch` なので dedup もキャッシュも効かず、再マウントのたびに再送されます。

原因は `BookmarkButton` を「`postId` さえ渡せばどこでも動く自己完結コンポーネント」として設計したことで、呼び出し側が状態取得を気にしないでいい代わりに一覧の文脈でのバッチ化ができなくなりまし

た。

正しくは一覧 API のレスポンスに `isBookmarked` / `isFollowing` を含めて prop で渡し、`useEffect` フェッチは prop が無い場合のフォールバックに格下げする。サーバ側では取得ユーザーが分かるので JOIN 1回で済みます。

Q23. `NEXT_PUBLIC_API_URL` をビルド引数で渡していますね。1つのイメージを staging と本番で使い回せませんか？

該当箇所: `nokoroa-frontend/Dockerfile` / `.github/workflows/deploy.yml`

回答

使い回せません。`NEXT_PUBLIC_` は `next build` 時に JS バンドルヘインライン展開され、公式も「if you build and deploy a single Docker image to multiple environments, all `NEXT_PUBLIC_` variables will be frozen with the value evaluated at build time」と明記しています。

今回は環境が prod 1つだけなのでビルド時固定の単純さを取りました。代償は、環境が増えた瞬間に環境ごとのビルドが必要になり、「staging で検証したイメージをそのまま本番に昇格する」という一般的なりリリース手順が取れなくなることです。

公式が示す逃げ道は「set up your own API to provide them to the client」で、Route Handler で `/api/config` を生やしてクライアントが起動時に読む形。あるいは同一ドメイン運用に倒して API を相対パスにすれば、そもそも環境変数にする必要がなくなります。

根拠: [Next.js — Environment Variables](#)

Q24. GitHub Actions が長期 AWS アクセスキーを持っていますね。OIDC がベストプラクティスでは？

該当箇所: `.github/workflows/deploy.yml`

回答

◆ おっしゃるとおりで、現状のままで指摘を受ける箇所です。GitHub Secrets に無期限の IAM ユーザーキーが常駐しています。

正しくは IAM OIDC Provider を作り、`sub` を `repo:<owner>/<repo>:ref:refs/heads/main` に絞った信頼ポリシーのロールを用意して、ジョブに `permissions: { id-token: write, contents: read }` を付け `role-to-assume` に切り替える。同 action は OIDC 対応済みなので差分は数行で、しかも main 以外のブランチや fork PR からは assume できなくなるので権限の粒度も上がります。

コスト対効果は明らかに OIDC 側です。優先度を落としたのは判断ミスでした。

根拠: [GitHub Actions — OpenID Connect](#)

Q25. FastAPI のストリーミングが同期ジェネレータですね。イベントループは大丈夫ですか？

該当箇所: `nokoroo-ai/app/routers/chat.py`

回答

イベントループはブロックしません。Starlette は `StreamingResponse` に同期ジェネレータを渡すと `iterate_in_threadpool` でワーカースレッドに逃がします。埋め込み側では明示的に `asyncio.to_thread` を使っていて、方針は一貫しています。

ただし問題は別にあって、◆ **スレッドプールの枠を長時間占有すること**です。1チャットが生成完了まで数十秒スレッドを保持するので、AnyIO の既定スレッド上限（40）に対して同時 40 チャットで頭打ちになります。backend 側のスロットルはユーザー単位なので同時接続数の上限にはなりません。

正しくは `google-genai` の `async` クライアントを使って `async def generate()` + `async for` にすること。そうすればスレッドではなくコルーチンなので、同時接続数はメモリの許す限りスケールします。

根拠: [FastAPI — Concurrency and async/await](#)

付録A — この資料を作る過程で見つけて直したバグ

面談で「レビューで何か見つかりましたか」と聞かれたら、この一覧が答えになる。

いずれも公式ドキュメントとの突き合わせ、または実測で発見している。

#	症状	原因	検証方法		
1	<code>PUT /users/profile</code> で現在のパスワード無しにパスワードを変更できた (アカウント乗っ取り)	DTO に <code>password / email</code> があり、サービスが DTO をそのまま Prisma の <code>data</code> に渡していた	OWASP API3:2023 との突き合わせ		
2	<code>PrismaService</code> が3インスタンス生成され接続プールが3本	<code>@Global()</code> な <code>PrismaModule</code> があるのに2モジュールが <code>providers</code> で再宣言	NestJS の DI スコープ仕様		
3	<code>AuthenticatedRequest</code> が Express の型ではなく Fetch API の <code>Request</code> を継承していた	<code>Request</code> を <code>@nestjs/common</code> (=デコレータの値) から import	tsc で実測 (<code>r.params</code> がエラー、 <code>r.bodyUsed</code> が通る)		
4	トーストが一切表示されていなかった	6ファイルが <code>react-hot-toast</code> を使うのに <code><Toaster /></code> が未マウント	grep で <code><Toaster</code> が 0 件		
5	同一 NAT 配下の別ユーザーが巻き添えで 429	グローバルの IP 単位ガードがコントローラの <code>@Throttle(20/分)</code> も読んでいた	実 DB で e2e 実測 (A が 21 回目で 429 → B が即 429)		
6	緯度0 (赤道)・経度0が <code>null</code> に潰れる	<code>\</code>	<code>\</code>	、で既定値に落としていた	コード読解
7	ベクトル検索の <code>MAX_LIMIT=50</code> が実際には 40 までしか返らない	<code>hnsw.ef_search</code> 既定 40 を超える契約	pgvector 公式ドキュメント		
8	長文投稿の後半が検索に効かない	<code>MAX_TEXT_LEN=8000</code> がモデルの 2,048 トークン上限を超過	Gemini API 公式ドキュメント		
9	チャットのタイピング中に絵文字が化ける	<code>split('')</code> が UTF-16 コードユニット単位でサロゲートペアを分断	コード読解		
10	地図の InfoWindow に未修正の HTML 文字列が残存	外部 API (ipapi.co) の応答をテンプレートリテラルで HTML に埋めていた	grep		
11	Google 連携のみのユーザーがパスワード変更で 500	<code>password</code> が <code>null</code> の状態で <code>bcrypt.compare(x, null)</code> が throw	schema の <code>password String?</code>		

付録B — 認識しているが未対応（◆）

聞かれたら即答できるようにしておく。聞かれなくても、終盤に自分から1〜2個出すと効く。

セキュリティ

項目	なぜ後回しにしたか	正しい直し方
JWT を localStorage に保管	Cookie 化は CSRF 対策・CORS・OAuth コールバックの作り直しまで一続きになる	短命アクセストークン（メモリ） + httpOnly refresh Cookie + CSRF トークン
CSP 未設定	Google Maps と MUI のインラインスタイルに依存し nonce 化が別作業	Middleware で nonce 発行 → <code>AppRouterCacheProvider</code> に渡す
トークン失効機構なし	Redis を増やす判断をせず、全リクエストの DB 往復も避けた	<code>user.tokenVersion</code> を payload に入れて <code>validate</code> で突き合わせ
OAuth コールバックで JWT を URL クエリに載せる	localStorage 方式の下流症状	一回限りの交換コード → <code>POST /auth/exchange</code> 、または Cookie 化
ログイン失敗カウンタが IP 単位	未認証なので IP しか鍵にできないと考えた	アカウント単位の失敗カウンタ + 段階的な遅延（ロックは DoS を生む）
<code>imageUrl</code> に内部宛先（127.0.0.1 / IMDS）を書ける	サーバ側で取得する経路が無いので SSRF にならない	許可リストではなく拒否リスト方式（既存データを壊さない）

設計・実装

項目	現状	正しい直し方
<code>tsconfig</code> が <code>strict: true</code> を自分で打ち消している	<code>strictNullChecks: false</code> / <code>noImplicitAny: false</code>	<code>strictNullChecks</code> を最優先で戻す（他の型バグの大半がこれに起因）
Prisma 生成型を使わず手書き <code>interface + as</code> 8箇所	スキーマ変更が型に追従しない	<code>satisfies</code> + <code>Prisma.PostGetPayload<></code>
<code>posts.service.ts</code> 710 行 / Haversine 3重複	関心事が4つ混在	サービス分割 + <code>Prisma.sql</code> で WHERE 句を共有
<code>@Res()</code> で Nest の抽象から降りている	上流 SSE をパススルーしなかった	<code>@Sse()</code> + <code>Observable</code> 、最低限 <code>res.on('close')</code> で上流を中断
<code>ThrottlerStorage</code> がインメモリ	1タスク運用なので顕在化しない	ElastiCache（Redis ストレージ）
クライアント側 N+1（一覧12件 → 12リクエスト）	コンポーネントの自己完結性を優先	一覧レスポンスに <code>isBookmarked</code> を同梱
<code>getOrCreateTags</code> の N+1 と競合	—	<code>createMany({ skipDuplicates: true })</code> + <code>findMany</code> の2クエリ
タグ更新がトランザクション外	—	<code>\$transaction</code> （外部 I/O は外に出す）

運用・インフラ

項目	現状	正しい直し方
デプロイに <code>prisma migrate deploy</code> が無い	CI のテスト DB にしか当たっていない	ECS デプロイ前に <code>aws ecs run-task</code> でワンショット実行
AI サービスのイメージが CD から漏れている	手動 build/push	<code>deploy-ai</code> ジョブ追加 + フォールバックイメージを削除
GitHub Actions が長期 AWS キー	優先度を落とした（判断ミス）	OIDC + <code>role-to-assume</code>
本番 PG 15.15 / ローカル PG 16	<code>engine_version</code> の既定値を上書きしていない	揃える（ <code>iterative_scan</code> は PG 16.5+ が必要）
<code>CREATE EXTENSION vector</code> がマイグレーションに埋まっている	RDS では <code>rds_superuser</code> が必要	インフラ側（Terraform）で先に張る
埋め込みモデル変更時の再埋め込みが手順書止まり	—	<code>post_embedding</code> に <code>model</code> 列を持たせ、不一致行を検索対象外に

付録C — 数字で答えられるようにしておく

項目	値
単体テスト	108 件
e2e テスト	54 件（実 DB / pgvector コンテナ）
埋め込み次元	768（pgvector のインデックス上限 2,000 に収める）
<code>hnsw.ef_search</code>	40（既定値、未設定）
ベクトル検索の上限	40 件
埋め込み入力長	2,000 文字（モデル上限 2,048 トークン）
RAG のコンテキスト	上位5件 × 600 文字
チャット履歴の転送上限	20 件 / 合計 20,000 文字
レート制限	全体 100/分（IP）、ログイン・登録 5/分（IP）、チャット 20/分（ユーザー）
JWT 有効期限	24 時間（失効機構なし）
JWT 鍵の最小長	32 文字（HS256 の出力長 256 bit に対応）

付録D — 参照した公式ドキュメント

Prisma / PostgreSQL / pgvector

- [Prisma — Raw queries](#)
- [Prisma — Operating against partial structures of model types](#)
- [Prisma — Transactions](#)
- [Prisma — CRUD](#)
- [Prisma — Development and production workflows](#)
- [Prisma — Unsupported database features](#)
- [PostgreSQL 16 — Frontend/Backend Protocol](#)
- [PostgreSQL 16 — CREATE EXTENSION](#)
- [PostgreSQL 16 — Unique Indexes](#)
- [pgvector — README / Filtering](#)

NestJS

- [Request lifecycle](#) / [Guards](#) / [Pipes](#)
- [Exception filters](#) / [Validation](#)
- [Rate limiting](#) / [Helmet](#)
- [Passport recipe](#) / [Custom decorators](#)
- [Controllers](#) / [Server-Sent Events](#)
- [Configuration](#) / [@nestjs/jwt](#)

OWASP

- [API Security Top 10 2023](#) ([API1](#) / [API3](#) / [API4](#))
- [HTML5 Security](#) / [JWT](#) / [Authorization](#) / [Authentication](#)
- [CSRF Prevention](#) / [DOM based XSS Prevention](#) / [SSRF Prevention](#)
- [LLM01:2025 Prompt Injection](#)

Next.js / Gemini / AWS / その他

- [Next.js](#) — [Server and Client Components](#) / [Environment Variables](#) / [headers\(\)](#)
- [Gemini API](#) — [Embeddings](#) / [Deprecations](#) / [Models](#)
- [AWS](#) — [Fargate task networking](#) / [ALB X-Forwarded-For](#)
- [AWS](#) — [RDS pgvector 0.8.0](#)
- [GitHub Actions](#) — [OpenID Connect](#)
- [Express](#) — [Behind proxies](#)
- [FastAPI](#) — [Concurrency and async/await](#)
- [WHATWG HTML](#) — [Server-sent events](#)
- [SWR](#) — [Mutation](#)